

Capstone (Full Stack Programming)

Problem Statement:

Pizza Store:

Pizzeria is an application where we can order the pizza of our choice.

There are 2 users in the application

1. Admin
2. Customer

##Admin Stories

1. As an admin, I should be able to login and logout into the application
2. As an admin, I should be able to do CRUD on the items available in the menu
3. As an admin, I can accept or reject the order form the user/ users
4. As an admin, I should be able send the message or pop up to users on order status
5. As an admin, I should be able to generate the bill of particular user
6. As an admin, I should be able to see the monthly revenue of the shop.

##User Stories

1. As an user, I should be able to register, login, and logout into the application
2. As a user, I should be able to see the different categories of the menu like pizza, sides, beverages, combo, new launches, bestsellers.
3. As a user, I should be able to select more than one item from the same category or different category.
4. As an user, I should be able to place / cancel the order
5. Once the order is placed I should be able to receive message or pop up on the order the status
6. As a user, I should be able to see the bill amount for the order along with payment options.

Proprietary Content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited

7. As a user, I should be able to see the mode of delivery.

Reference:

Please refer the link <https://www.dominos.co.in/>

Sprint Plan:

Sprint I Objectives:

1. Create database schema (along with their relationships)
2. CRUD on User and Admin (login, logout, register)
3. Create template using React

Sprint II Objective:

1. Develop search functionality based on different criteria.
2. The menu items CRUD for Admin
3. Create / cancel order from user
4. Implement billing option (generate for Owner and view for User)
5. Implement Spring security, JWT
6. Implement component and end-to-end testing

Sprint III Objective:

1. Create Data Transfer Objects
2. Create Service Layer logic
4. Create controller to direct rest API
5. Create message service for order status
6. Create payment mode service
7. Integrate FrontEnd with BackEnd

Proprietary Content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited

8. Add extra features
9. Make application responsive

Instructions:

1. Set Up MongoDB Database:
 1. Install MongoDB on your system if not already installed.
 2. Create a MongoDB database to store application data.
 3. Design appropriate collections to store user and application information.
2. Develop Backend with Node.js and Express.js:
 1. Create a Node.js project.
 2. Set up Express.js to handle routing and middleware.
 3. Implement APIs for user authentication (register, login, logout) and CRUD operations for applications.
 4. Ensure APIs are RESTful and follow best practices.
 5. Interact with MongoDB.
3. Develop Frontend with React:
 1. Set up an React project.
 2. Create components for login, registration, application listing, application details, user profile, etc.
 3. Implement routing to navigate between different components.
 4. Integrate HTTP client module to communicate with the backend APIs.
 5. Use React Material or Bootstrap for UI components to ensure better UI.
4. Make Application Responsive:
 1. Use CSS media queries and flexible layout techniques to ensure the application is responsive across different devices and screen sizes.
5. Handle Session using Local Storage:
 1. Implement session management using JSON Web Tokens (JWT).
 2. Store JWT tokens in the local storage to maintain user sessions.
6. Implement Proper Component Structure:
 1. Organize React components, services, and modules following best practices and maintainability.

Proprietary Content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited

7. Include Client-Side Validation:

1. Implement client-side form validation using React forms and validators to ensure data integrity and improve user experience.

8. Perform Component and End-to-End Testing:

1. Write unit tests for React components and services using proper testing framework.
2. Conduct end-to-end testing using a testing framework to ensure the application works as expected from the user's perspective.

9. Create a Single Page Application (SPA):

1. Ensure the application functions as a single-page application where navigation occurs without refreshing the entire page.
2. Utilize React's routing capabilities to achieve this behavior.